



GRUPO 3 · REQUERIMIENTOS FUNCIONALES

Pagos de Servicios

Módulo: Pago de facturas (ANDE, ESSAP, telefonía)

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 3
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aikuaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

Documento derivado del código fuente de aikuaa-sandbox-api y aikuaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre la consulta de facturas de servicios de un titular y el pago de una factura pendiente. El pago es la única operación del sandbox que ejecuta dos escrituras dentro de una misma transacción: registra el pago y marca la factura como pagada, todo o nada.

Fuera de alcance: Emisión de facturas (los datos se cargan por el sembrado), cálculo de intereses o recargos por mora, pagos parciales, anulación o reverso de un pago, y la validación real del medio de pago elegido.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de usuarios)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos <code>qa_api</code> (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy <code>/api/proxy/{path}</code> ; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET <code>/api/v1/facturas</code>	RF-G3-01	Lista facturas, con filtros por titular y estado.
GET <code>/api/v1/facturas/{id}</code>	RF-G3-02	Devuelve el detalle de una factura.
POST <code>/api/v1/facturas/{id}/pagar</code>	RF-G3-03	Paga una factura pendiente y registra el pago.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key".
- Cada API key admite un máximo de **30 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE`, y las operaciones que "eliminan" hacen baja lógica (`activo = false`).
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.

- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Los escenarios de pago necesitan al menos una factura en estado `pendiente`; los datos sembrados incluyen facturas pendientes, pagadas y vencidas de los proveedores ANDE, ESSAP, COPACO, Tigo y Personal.
- Una factura solo se puede pagar una vez: los escenarios que repiten el pago deben usar una factura pendiente distinta o esperar el error de la segunda llamada.

3. Modelo de datos del módulo

facturas

Facturas de servicios emitidas a un titular. El módulo las lee y actualiza su estado al pagarlas.

Columna	Tipo / restricción
id	bigserial, clave primaria
usuario_id	bigint, obligatorio, referencia a usuarios(id)
proveedor	text, obligatorio, uno de ANDE / ESSAP / COPACO / Tigo / Personal
numero_factura	text, obligatorio y único
monto	numeric(12,2), obligatorio
fecha_vencimiento	date, obligatorio
estado	text, uno de pendiente / pagada / vencida; por defecto pendiente
created_at	timestampz, por defecto now()

pagos

Pagos aplicados a facturas. El módulo inserta una fila por cada pago aceptado.

Columna	Tipo / restricción
id	bigserial, clave primaria
factura_id	bigint, obligatorio, referencia a facturas(id)
usuario_id	bigint, obligatorio, referencia a usuarios(id)
monto	numeric(12,2), obligatorio
metodo_pago	text, uno de tarjeta / cuenta / efectivo
estado	text, uno de procesado / fallido / pendiente; por defecto procesado
created_at	timestampz, por defecto now()

El importe del pago no lo elige quien paga: el sistema lo toma de la factura. Tampoco se elige el titular del pago, que se copia del titular de la factura. Una factura en estado `vencida` sí puede pagarse: la única condición es que no esté ya `pagada`.

4. Requerimientos funcionales

RF-G3-01 — Listar facturas

GET /api/v1/facturas

Devuelve las facturas del sandbox, con filtros combinables por titular y por estado, para ubicar las facturas que quedan por pagar.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): uno de `pendiente`, `pagada` o `vencida`. Cualquier otro valor es rechazado.

Reglas de negocio

- Los dos filtros son opcionales y se combinan con Y lógico: enviando ambos se obtienen las facturas de ese titular en ese estado.
- El resultado se ordena por identificador ascendente y está limitado a 100 registros, con o sin filtros.
- Una combinación de filtros sin coincidencias devuelve una lista vacía, no un error.

Respuesta esperada

- `200 OK` con `data` como arreglo de facturas; cada elemento incluye `id`, `usuario_id`, `proveedor`, `numero_factura`, `monto`, `fecha_vencimiento`, `estado` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	estado con un valor fuera de la lista permitida, o <code>usuarioId</code> no numérico.
UNAUTHORIZED	401	Falta la API key o es inválida.

Fuente: `app/api/v1/facturas/route.ts`

RF-G3-02 — Consultar una factura

GET /api/v1/facturas/{id}

Devuelve el detalle de una factura puntual: proveedor, número, importe, vencimiento y estado actual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la factura cuyo identificador coincide exactamente, sin importar su estado.
- Si no existe, responde 404 con el mensaje "Factura no encontrada."

Respuesta esperada

- `200 OK` con `data` conteniendo la factura completa.

Errores

Código	HTTP	Cuándo ocurre
NOT_FOUND	404	No existe una factura con ese identificador.
VALIDATION_ERROR	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/facturas/[id]/route.ts`

RF-G3-03 — Pagar una factura

POST /api/v1/facturas/{id}/pagar

Paga una factura que aún no fue pagada: registra el pago con el medio elegido y marca la factura como pagada. Ambas escrituras ocurren dentro de una misma transacción.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la factura.
- `metodoPago` (obligatorio, en el cuerpo): uno de `tarjeta`, `cuenta` o `efectivo`.

Reglas de negocio

- Solo se puede pagar una factura que no esté en estado `pagada`. Una factura `pendiente` o `vencida` es pagable.
- La factura se bloquea durante la operación, de modo que dos pagos simultáneos de la misma factura no puedan prosperar los dos: uno gana y el otro encuentra la factura ya pagada.
- El importe del pago se copia del importe de la factura; el cliente no puede pagar de más ni de menos, ni hacer pagos parciales.
- El titular del pago se copia del titular de la factura.
- El pago se registra siempre con `estado = 'procesado'`: no hay medio de pago que pueda rechazarlo.
- Acto seguido, y dentro de la misma transacción, la factura pasa a `estado = 'pagada'`. Si algo falla, ninguna de las dos escrituras queda aplicada.
- Una factura ya pagada no se distingue de una inexistente: ambas responden 404 con el mismo mensaje.
- **No se valida el vencimiento, ni la existencia de fondos, tarjeta o cuenta asociada al medio de pago elegido.**

Respuesta esperada

- `200 OK` con `data` conteniendo dos objetos: `factura` (ya con `estado = 'pagada'`) y `pago` (con `id`, `factura_id`, `usuario_id`, `monto`, `metodo_pago`, `estado` y `created_at`).

Errores

Código	HTTP	Cuándo ocurre
NOT_FOUND	404	La factura no existe o ya está pagada (mensaje: "Factura no encontrada o ya pagada.").
VALIDATION_ERROR	400	<code>metodoPago</code> ausente o con un valor fuera de la lista permitida.
EXECUTION_ERROR	400	Fallo de base de datos al registrar el pago o actualizar la factura.

Fuente: `app/api/v1/facturas/[id]/pagar/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body".
- **Errores de base de datos:** una violación de restricción (CHECK, UNIQUE, clave foránea) no aborta el servicio: se traduce a `400 EXECUTION_ERROR` con el mensaje que devuelve PostgreSQL.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.

Códigos de error

Código	HTTP	Significado
UNAUTHORIZED	401	Falta el header <code>x-api-key</code> , o la clave no existe o está inactiva.
RATE_LIMITED	429	Se superaron las 30 requests por minuto de esa API key.
VALIDATION_ERROR	400	La entrada no cumple el esquema de la ruta, o el JSON es inválido.
EXECUTION_ERROR	400	La consulta falló en la base de datos (restricción violada, tipo inválido).
NOT_FOUND	404	El recurso indicado no existe, o no está en un estado que admita la operación.
INTERNAL_ERROR	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
/facturas — Listado de facturas	Tabla con proveedor, número, importe, vencimiento y estado, con filtros por titular y por estado. Ejecuta RF-G3-01.
/facturas/{id} — Detalle de factura	Muestra los datos de la factura y, cuando corresponde, el bloque "Pagar factura" con el selector de medio de pago. Ejecuta RF-G3-02 y RF-G3-03.

- Tras un pago exitoso la pantalla de detalle actualiza la factura en pantalla con el estado devuelto por la API, sin necesidad de recargar.
- El selector de medio de pago ofrece exactamente las tres opciones aceptadas por la API, de modo que el error de valor inválido solo puede provocarse llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G3-01 — Listar facturas

- Al listar facturas filtrando por `estado=pendiente`, todos los elementos devueltos tienen ese estado.
- Al combinar `usuarioId` y `estado`, todos los elementos cumplen ambas condiciones simultáneamente.
- Al filtrar por `estado=anulada`, la respuesta es 400 `VALIDATION_ERROR` porque el valor no pertenece a la lista permitida.
- Al filtrar por un titular sin facturas, la respuesta es 200 con `data` vacío.

RF-G3-02 — Consultar una factura

- Dada una factura existente, al consultarla la respuesta es 200 y `data.numero_factura` coincide con el de la factura.
- Al consultar una factura inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Factura no encontrada".
- Después de pagar una factura, al consultarla nuevamente su `estado` es `pagada`.

RF-G3-03 — Pagar una factura

- Dada una factura pendiente, al pagarla con un medio válido la respuesta es 200, `data.factura.estado` es `pagada` y `data.pago.estado` es `procesado`.
- El importe de `data.pago.monto` es igual al importe de la factura, sin que el cliente lo haya enviado.
- El `usuario_id` del pago coincide con el titular de la factura.
- Al pagar por segunda vez la misma factura, la respuesta es 404 `NOT_FOUND` con el mensaje "Factura no encontrada o ya pagada.", y no se registra un segundo pago.
- Al pagar una factura en estado `vencida`, la operación es aceptada y la factura queda `pagada`.
- Al pagar con `metodoPago=cripto`, la respuesta es 400 `VALIDATION_ERROR` y la factura conserva su estado anterior.
- Al pagar una factura inexistente, la respuesta es 404 y no se crea ninguna fila en `pagos`.

8. Anexo normativo (BCP / SEDECO) y brechas

Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1334/98 de Defensa del Consumidor (SEDECO), constancia de pago	El consumidor debe recibir comprobante del pago con importe, fecha y medio utilizado.	Cubierto: la respuesta devuelve el pago completo con importe, medio y fecha, y la web lo refleja en el detalle.
Ley 1334/98, prohibición de cobros indebidos	No debe cobrarse dos veces el mismo concepto ni un importe distinto al facturado.	Cubierto: el importe se toma de la factura y el bloqueo transaccional impide el doble pago de la misma factura.
Marco del BCP sobre medios de pago electrónicos	El medio de pago debe validarse y la operación debe poder rechazarse si el instrumento no es válido o no tiene fondos.	No implementado: el medio de pago es una etiqueta; el pago siempre se registra como procesado.
Ley 4868/2013 de Comercio Electrónico, información previa a la transacción	El usuario debe conocer el importe exacto y las condiciones antes de confirmar el pago.	Cubierto: el detalle de la factura muestra importe y vencimiento antes de habilitar el pago.
Reglas de mora y recargos de los prestadores de servicios	Una factura vencida debería liquidarse con los recargos correspondientes.	No implementado: pagar una factura vencida se registra por el importe original, sin recargo.

Brechas conocidas del sandbox

- El pago no valida la fecha de vencimiento ni calcula recargos: una factura vencida se paga por el importe original.
- El medio de pago no se verifica contra ninguna tarjeta o cuenta del titular; nunca hay pagos **fallidos** ni **pendientes** generados por la API.
- No existen pagos parciales ni anulación o reverso de un pago ya registrado.
- Una factura ya pagada devuelve el mismo 404 que una inexistente, lo que impide distinguir ambas situaciones desde la respuesta.
- El estado **vencida** nunca se asigna automáticamente: no hay proceso que marque como vencidas las facturas cuyo plazo expiró.
- El listado está limitado a 100 registros sin paginación.